

TALLINN UNIVERSITY OF TECHNOLOGY
School of Information Technologies

Daniil Fetisov 223367IVSB

Sysmon Event Log Analysis Using Machine Learning

Bachelor's thesis

Supervisor: Margus Ernits
Msc.

Tallinn 2025

TALLINNA TEHNIKAÜLIKOOL
Infotehnoloogia teaduskond

Daniil Fetissov 223367IVSB

Sysmon sündmuslogide analüüs, kasutades masinõppetehnoloogiaid

Bakalaureusetöö

Juhendaja: Margus Ernits
Msc.

Tallinn 2025

Author's declaration of originality

I hereby certify that I am the sole author of this thesis. All the used materials, references to the literature and the work of others have been referred to. This thesis has not been presented for examination anywhere else.

Author: Daniil Fetissov

12.05.2025

Abstract

This thesis explores an application of machine learning techniques for Sysmon event log analysis. The primary objective is to gain insights into required architecture to implement effective supervised machine learning model that can accurately detect malicious activity in the environment.

The study explores controlled Sysmon log dataset acquisition using synthetically generated activity, its labelling and preprocessing, model training and its evaluation. Open-source frameworks are used for activity generation and models trained include Random Forest classifier and LSTM neural network.

Research highlights advantages and disadvantages of the machine learning-based approach to detection engineering, as well as provides discussions on rule-based vs. machine learning based detection engineering and the role of synthetic data.

The work concludes with author offering recommendations for future work and providing insights into encountered challenges, as well as summarizing findings in the final chapter.

This thesis is written in English and is 49 pages long, including 7 chapters, 7 figures and 3 tables.

List of abbreviations and terms

ACM SIGSAC	Association for Computing Machinery Special Interest Group on Security, Audit and Control
ART	Atomic Red Team
ATT	Adversarial Tactics, Techniques
CMU-SEI	Carnegie Mellon University- Software Engineering Institute
DNS	Domain Name System
EVTX	Event Log File Extension
GDPR	General Data Protection Regulation
GHOSTS	Generic Host and Orchestrated Simulation Tool
GUID	Globally Unique Identifier
HIPAA	Health Insurance Portability and Accountability Act
ID	Identifier
IP	Internet Protocol
IT	Information Technology
LTSM	Long Short-Term Memory
ML	Machine Learning
NIST	National Institute of Standards and Technology
RF	Random Forest
ROC-AUC	Receiver Operating Characteristic - Area Under Curve
SIEM	Security Information and Event Management
SVM	Support Vector Machine
UEBA	User and Entity Behavior Analytics
VBA	Visual Basic for Applications
VM	Virtual Machine
XAI	Explainable Artificial Intelligence
XML	eXtensible Markup Language

Table of contents

1 Introduction	10
1.1 Problem statement	10
1.2 Sysmon Background.....	11
1.3 Objectives	12
1.4 Scope and Limitations	12
2 Research Foundations.....	14
2.1 Sysmon and Event Logging in Windows Systems	14
2.2 Importance of Event Logs for Security Monitoring	14
2.3 Rule-Based Detection vs. Machine Learning-Based Detection	15
2.4 Supervised and Unsupervised Machine Learning	16
2.5 Related Work: Automated Event Correlation in Cybersecurity	17
3 Analysis	18
3.1 Methodology.....	18
3.2 Rationale for Choosing Supervised Learning and Synthetic Data	19
3.3 Dataset Generation	19
3.4 Dataset Preprocessing.....	21
3.5 Model Selection.....	22
3.6 Model Evaluation	23
4 Implementation.....	25
4.1 Dataset generation	25
4.1.1 Benign Activity Generation.....	25
4.1.2 Malicious Activity Generation	26
4.1.3 Data Labeling	27
4.1.4 General Dataset Preprocessing	29
4.2 Models Training and Evaluation	30
4.2.1 Random Forest.....	30
4.2.2 LSTM.....	34
4.2.3 Results Interpretation.....	38
5 Discussion.....	39

5.1 Detection Engineering: Rule-Based vs Machine Learning	39
5.2 The Role of Synthetic Data Generation.....	41
5.3 Conclusion of Discussion	41
6 Limitations, challenges and future work	43
6.1 Limitations and Challenges	43
6.2 Future Work.....	44
7 Conclusion	45
References	47
Appendix 1 – Non-exclusive licence for reproduction and publication of a graduation thesis	49

List of figures

Figure 1. Distribution of Labels	28
Figure 2. Random Forest Confusion Matrix - Top 6 Features	31
Figure 3. Feature Importance Chart for Random Forest	32
Figure 4. Random Forest Confusion Matrix - Top 6 Features	33
Figure 5. Training Loss Reduction Over Epochs for LSTM Model	35
Figure 6. Precision-Recall Curve for LSTM Model.....	36
Figure 7. LSTM Confusion Matrix for Benign vs. Malicious Classification.....	37

List of tables

Table 1. Random Forest Classification Report – All Features	31
Table 2. Random Forest Classification Report – Top 6 Features.....	33
Table 3. LSTM Classification Report.....	36

1 Introduction

In modern cybersecurity, detailed system-level telemetry is essential for detecting and responding to advanced threats. Among various sources of such telemetry, Microsoft's System Monitor (Sysmon) stands out as a powerful tool for capturing low-level Windows activity, including process launches, file changes, and network connections. These logs are instrumental in tracing malicious behavior, especially in environments where traditional antivirus solutions and signature-based detections may fail.

However, the sheer volume and complexity of Sysmon logs make manual analysis impractical in most cases. A single endpoint may generate tens of thousands of events daily, and in enterprise settings, this scales to millions of logs across the network. Traditional rule-based detections are the current industry standard to overcome this issue. However, they rely on predefined patterns that can become outdated, leading to missed threats and/or excessive false positives - a phenomenon known as alert fatigue.

To address these challenges, the use of machine learning (ML) for log analysis is gaining popularity. ML models can learn from historical behavior, identify complex or novel patterns, and help reduce the noise in detection systems. This thesis explores how ML can be applied to Sysmon event logs to enhance threat detection, reduce false positives, and uncover patterns that rule-based systems may miss.

While Sysmon is now also available for Linux systems, this work focuses specifically on the Windows implementation, which remains the most mature and widely used in enterprise environments.

1.1 Problem statement

Modern endpoint environments generate overwhelming volumes of Sysmon telemetry, while adversarial behaviours continue to evolve and bypass static defences. This makes it increasingly difficult to build and maintain effective rule-based detections. At the same

time, the lack of labelled Sysmon datasets poses a challenge for developing machine learning-based solutions that could provide scalable, adaptive detection capabilities.

This raises following research question: What is the architecture that enables supervised machine learning models to be effectively developed and trained to detect malicious activity patterns that might evade traditional rule-based detection approaches, given the significant constraints of limited and often imbalanced real-world labelled datasets?

1.2 Sysmon Background

Sysmon was introduced as part of Microsoft's Sysinternals suite to enhance Windows monitoring for security purposes. Once installed on a host, it remains persistent across reboots, and it continuously logs a wide range of system events into the Windows Event Log's Sysmon channel. Sysmon captures detailed information that is not available in standard Windows security logs, such as parent process IDs, command-line arguments of processes, file hashes, and raw disk access events. This telemetry makes Sysmon a powerful tool for threat hunters and incident responders. By deploying Sysmon across endpoints, organizations gain deep visibility into host-level activities that may indicate cyberattacks [1]. For instance, Sysmon can record the execution of suspicious processes or scripts, the loading of drivers or DLLs, network connections to unusual destinations, and changes to registry or file timestamps - all of which provide invaluable information for detecting intrusions. Sysmon's role in cybersecurity has grown as advanced threats increasingly employ stealthy tactics. Its detailed logs help expose behaviours that default logs might miss, as evident by recent research [2].

In practice, Sysmon data is often forwarded to Security Information and Event Management (SIEM) platforms or log analytics systems, where it can be correlated and analysed along with other security telemetry to uncover attacks. Log analysis, in general, is a cornerstone of modern threat detection: event logs from systems and applications serve as an audit trail of activity, enabling investigators to reconstruct actions and identify anomalies. Logs provide detailed insights into the behaviour and state of IT systems, helping security professionals detect, prevent, and investigate incident.

1.3 Objectives

This thesis aims to research how a machine learning-based solution can enhance threat detection on the example of Sysmon event logs. The specific objectives include:

Data Collection and Preprocessing: Implement a solution that would allow for Sysmon event log dataset acquisition for model prototyping.

Machine Learning Implementation: Research suitable machine learning techniques for log analysis and implement a proof-of-concept model.

Model Evaluation: Assess the performance, accuracy, and other metrics of the developed model in log analysis scenarios.

1.4 Scope and Limitations

Scope:

- Collecting/generating Sysmon logs and pre-processing data for machine learning models.
- Researching and implementing suitable machine learning techniques to correlate Sysmon logs.
- Evaluating model performance.

Limitations:

- The project will be limited to Sysmon event logs on Windows systems and will not cover other types of logging systems.
- Due to the time constraint, the thesis may not provide exhaustive comparison across all machine learning models; instead, the focus will be on a single/few selected, relevant approaches.
- The model's scope is limited to a proof of concept rather than a fully integrated deployment-ready solution. Further optimizations and integrations with other infrastructure fall outside this project scope.

- The threat detection goal will be focused on detecting single/few chosen MITRE ATT&CK technique/s, as author is limited by computational resources as well as the time constraint.
- The dataset used for training will not be acquired from a real enterprise environment, rather the goal would be to acquire one that would resemble real enterprise activity as close as possible, as author was unable to find an openly available real-world labelled dataset of Sysmon logs suitable for this study.

2 Research Foundations

This chapter provides the theoretical background for understanding Sysmon event logging, their significance in security monitoring, and the role of machine learning in analyzing these logs. It includes an overview of event logging in Windows systems, the advantages and limitations of rule-based detection versus machine learning approaches, and key machine learning techniques relevant to log correlation.

2.1 Sysmon and Event Logging in Windows Systems

Windows event logs serve as an important part of system security analysis. These logs contain detailed records of system and application activities, providing insight into potential security threats and emerging incidents.

Sysmon extends standard Windows event logging capabilities by capturing detailed telemetry on process creation, registry modifications, file changes, and network connections. By enriching log data with cryptographic hashes, parent-child process relationships, and IP address tracking, Sysmon enhances the ability to detect internal threats [1].

2.2 Importance of Event Logs for Security Monitoring

System and security event logs are fundamental to threat detection and incident response. They provide valuable insights into user activities, process execution, network connections, and file system changes. Organizations use logs to:

- **Detect Unauthorized Access:** Unauthorized login attempts, privilege escalation, and lateral movement within a network can be identified.
- **Investigate Security Incidents:** Event logs help in reconstructing the sequence of events leading up to a breach or attack.

- **Ensure Compliance:** Regulations like GDPR, HIPAA, and NIST require log monitoring to track security incidents and ensure accountability.
- **Enable Threat Hunting:** Security analysts proactively search for threats by analyzing anomalies and correlating suspicious events.
- **Ensure Adherence to Internal Policies:** Disallowed applications, connections and other internal policy violations can be tracked via event logs.

2.3 Rule-Based Detection vs. Machine Learning-Based Detection

Traditional rule-based detection relies on analyst-crafted rules to identify malicious patterns. Even though static rules have proven to be an extremely effective detection strategy, they come with several major disadvantages. Shortcomings include limited adaptability to new threats, the necessity for frequent manual updates by skilled human analysts, and a high rate of false positives leading to alert fatigue among security analysts. For instance, overly broad rules can generate numerous false alarms, while overly narrow rules risk missing actual security incidents. Achieving the right balance requires multiple iterations of fine tuning, which takes a lot of time and manual work of an experienced specialist.

In contrast, machine learning (ML)-based detection offers adaptability and improved threat recognition capabilities. ML approaches learn patterns directly from data, allowing them to generalize beyond known examples and automatically adapt to new threats without manual intervention, significantly reducing human maintenance. ML-based systems are far better at identifying complex, subtle correlations and combinations of indicators indicative of malicious activities, something challenging to accomplish with static rules. Supervised ML is particularly effective because it learns directly from labelled examples, thus providing clear advantages over traditional rule-based systems in terms of precision and flexibility, however labelling real data precisely without human interaction is a big challenge and requires clever solution in place, something that does not exist in the perfect form yet.

Machine learning approaches also come with inherent challenges. Supervised ML models depend heavily on high-quality labelled datasets, which can be difficult to acquire as already mentioned or they can be even susceptible to labelling inaccuracies due to human

error. Additionally, certain ML models - particularly those based on deep learning - often operate as "black boxes", making their decision-making processes difficult to interpret, a huge drawback in security contexts where understanding detection reason is crucial for further improvements. Moreover, without careful validation, ML models can become overly specialized to their training datasets, known as overfitting, which impacts their ability to generalize effectively to new, real-world scenarios.

Despite these challenges, the scalability and automation advantages provided by ML methods remain highly beneficial, particularly for handling extensive log data volumes, such as those produced by Sysmon in modern environments. Evidence to that is the amount of recent academic and big security vendor sponsored research on the topic of automated log analysis, that will be discussed later in this chapter.

2.4 Supervised and Unsupervised Machine Learning

Supervised learning utilizes labelled data, training algorithms on examples explicitly annotated with correct outcomes. It is particularly effective for tasks like classifying known malicious behaviours, as the model learns clear distinctions between benign and malicious activities, ensuring precise detection of threats [3].

Unsupervised learning, on the other hand, operates without labelled data, detecting patterns or anomalies within unlabelled datasets. It is useful in identifying previously unknown threats but typically has lower precision, often leading to more false positives due to the general nature of anomaly detection [4].

Both methods have relevance for Sysmon log analysis, however, supervised learning will be the focus of this work with justifications for it provided in the next chapter.

Several supervised ML algorithms are suitable for log correlation, including Decision Trees, Random Forests, Support Vector Machines (SVM), and Neural Networks. Decision trees provide high interpretability and ease of understanding, making them appealing for security analysts, although they risk overfitting if overly complex. Random Forest algorithms were shown to offer enhanced performance in intrusion detection systems by effectively handling large datasets and reducing overfitting issues [5]. SVMs were shown to be effective at classification tasks where clear margin separations existed but faced scalability challenges when handling extensive log datasets. They have

demonstrated high accuracy in detecting various types of network intrusions, making them a valuable tool in cybersecurity applications [6].

Neural networks, including deep learning models like Long Short-Term Memory (LSTM) networks, offer advanced capabilities in modelling temporal log sequences and capturing intricate event patterns, although they require more computational resources and large datasets for training. These networks are particularly suited to capturing complex temporal relationships among sequences of events, making them effective in detecting new threats [7]. Supervised learning techniques can also be effectively adapted for anomaly detection by training models to recognize normal behaviour and flagging deviations as anomalies, thus capturing both known and novel threats [3].

2.5 Related Work: Automated Event Correlation in Cybersecurity

Academic research extensively explores ML applications in log analysis. Levshun and Kotenko (2023) provide a comprehensive survey of artificial intelligence approaches for security event correlation, highlighting ML methods as particularly promising due to their adaptability and accuracy [8]. Smiliotopoulos et al. demonstrated supervised ML's effectiveness in detecting lateral movement within Windows Sysmon logs, achieving over 99% accuracy using diverse algorithms ranging from classical methods to deep learning techniques [9]. Du et al. developed DeepLog, employing deep neural networks to detect anomalies by learning log event sequences, advancing automated log analysis [7]. These works collectively demonstrate ML's potential in automatically capturing temporal and contextual relationships within log data.

The cybersecurity industry has also adopted ML for automated log correlation. Elastic Security integrates supervised ML models to analyse Windows Sysmon logs, identifying suspicious activities and reducing reliance on predefined static rules [10]. User and Entity Behaviour Analytics (UEBA) solutions in SIEM systems, such as ManageEngine's EventLog Analyzer, utilize ML to detect behavioural anomalies indicative of insider threats or account compromise by modelling normal behaviour and identifying deviations [11].

These are just a few examples; in reality new solutions are developed daily and the industry advances at a very rapid pace.

3 Analysis

Effective threat detection using Sysmon logs requires a structured approach, especially given the scarcity of labelled public datasets and the complexity of event correlations. To address this, a custom experimental framework will be developed, combining simulated data generation, preprocessing, and machine learning-based classification.

A custom dataset will be generated through controlled simulations of both benign and malicious activity. After collection, the data will be preprocessed to create a structured format suitable for machine learning tasks. Machine learning models will then be selected to represent distinct approaches to log analysis: one treating events independently, and the other capturing sequential patterns.

The following sections describe in detail the methodology used, reasoning behind choosing supervised learning and synthetic data generation, the data collection and preprocessing processes, the model selection, and the evaluation metrics used to assess detection performance.

3.1 Methodology

The methodology adopted in this thesis follows a structured framework designed to address the key challenges in detecting malicious activity through Sysmon logs. This framework is composed of four main stages:

Dataset Generation – A synthetic dataset is created using automated tools to simulate benign user activity and malicious behavior based on known adversary techniques. This step ensures full control over the environment and allows for reliable labeling of events.

Data Preprocessing – The raw logs generated are parsed, normalized, and transformed into a machine learning-ready format. This includes handling missing values, encoding categorical data, and structuring event fields consistently.

Model Selection and Training – Two distinct types of machine learning models are selected: a traditional classifier, which treats events independently, and a more complex neural network, which learns from event sequences. Each model is trained on the labeled dataset to learn patterns associated with benign and malicious activity.

Model Evaluation – The trained models are evaluated on a separate test set using standard classification. These metrics provide insight into how well each model performs in distinguishing between benign and malicious Sysmon events.

This methodology is designed to deliver a proof-of-concept implementation for automated threat detection using Sysmon logs, while also accounting for the practical constraints of limited computing resources and the absence of real-world labelled data.

3.2 Rationale for Choosing Supervised Learning and Synthetic Data

Supervised learning is chosen for its precision and effectiveness in classifying known malicious behaviors, significantly reducing false alarms, and providing clear, measurable evaluation metrics. Although unsupervised learning requires significantly less setup, it often lacks precision and necessitates substantial amounts of training data [4], challenging to obtain within a bachelor thesis's scope.

However, scarcity of real-world labeled log data for supervised training also presents challenges due to confidentiality and privacy concerns. Log generation based on synthetic activity addresses this gap by enabling comprehensive, controlled creation of labeled datasets. Datasets generated on synthetic activity offer extensive coverage of diverse attack scenarios, balanced training data, and allow proactive learning of emerging attack techniques [12]. Hence, supervised learning combined with synthetic log generation is optimal for Sysmon event log analysis within the scope of this work. Nevertheless, it is crucial to keep in mind when evaluating the end result of this practical part of the work, that synthetic data inherently simplifies real-world complexity, and models trained solely on it may exhibit optimistic performance that does not fully translate to live environments.

3.3 Dataset Generation

Because the author was unable to find publicly available benchmark dataset of Sysmon event logs with ground-truth labels, custom dataset will be generated using controlled

simulation. The approach will involve leveraging automated tools and scripts to produce Sysmon logs for both ordinary system behavior and malicious activities. In practice, this means setting up a Windows environment with Sysmon installed and then orchestrating two categories of behavior:

Benign activity simulation: Will simulate normal user and system operations to produce benign log events. This will include actions such as launching common applications, performing file operations, network browsing, and other typical processes that would appear in enterprise environments. An open-source automation framework will be configured to mimic human user behavior and routine system tasks (e.g. opening browsers, running office software, etc.), generating a rich variety of Sysmon events (Process Creation events, File Creation/Deletion, Registry access, Network connections that are typical, etc.). Each log entry produced in this phase will be implicitly labeled as "benign" in future dataset.

Malicious activity simulation: Known attack technique will be executed in the lab environment to produce Sysmon logs corresponding to malicious behavior. Using adversary simulation tools and custom scripts, selected attack/attacks will be triggered numerous times with random periods between them. These actions are designed to emulate real cyber threats and thus generate log events indicative of compromise. As the goal of the practical exercise is not a deployment-ready solution, rather a proof-of-concept model, the approach will focus on a small number/single specific adversary techniques, that better suit the built lab environment. All events generated during these simulated attacks will be labeled as "malicious" in the dataset based on the indicators logged by the used adversary activity generation tool.

This generation strategy ensures coverage of both benign and malicious events in the Sysmon log format.

The volume of events will be balanced so that the dataset will contain a substantial number of malicious examples relative to benign ones. In a real-world logging scenario, malicious events are very infrequent compared to benign events. However, training a model on a severely imbalanced dataset can lead to a bias toward predicting the majority class (benign). To address this, the malicious behaviors during data generation will be oversampled on purpose - in essence, attack simulations will be run numerous times or

with variations so that the final log set contains a reasonable proportion of malicious entries (while still reflecting more benign events overall). This approach yields a dataset where the model can learn what malicious events look like without being overwhelmed by the number of benign events. Additionally, reasoning behind this decision is caused by limited computational resources for model training, making it infeasible to firstly generate very large datasets and secondly ingest it into the model.

It is also vital to understand that although the data will be synthetic, it will be generated using real Sysmon logging of actual (simulated) activities, so the format and fields will be identical to what one would see on a real system's Sysmon log. Each log entry in the raw dataset will include the Sysmon Event ID (indicating the type of event, e.g., process creation is Event ID 1), a timestamp, and various event-specific fields (such as process name, process ID, parent process, command-line arguments, file path, network addresses, etc.), along with later added label denoting it as benign or malicious.

By following this approach, the dataset generation method will provide a controlled set of Sysmon logs for this study. The exact tools and configurations that will be used to generate the data (for example, specific automation frameworks for benign user behavior and the libraries of attack techniques for malicious behavior) will be described in Chapter 4, as this section focuses on the methodology rather than implementation details.

In summary, the expected end result of data generation is a labelled dataset of Sysmon events, containing a mixture of normal and malicious logs, which will serve as the foundation for training and evaluating machine learning models.

3.4 Dataset Preprocessing

Raw Sysmon log data, as collected in the previous step, is not immediately ready to feed into machine learning algorithms. Each Sysmon log entry is a semi-structured record (often in XML or EVTX format when collected), containing various fields depending on the event type. Before modeling, data preprocessing steps to clean and transform these logs into a structured dataset (e.g., a table of feature values) suitable for supervised learning is required.

As default Sysmon configuration captures too much unnecessary noise that can be filtered more easily on the capturing step, rather than preprocessing step, a slightly modified

version of a widely-used Sysmon configuration file will be employed, based on the openly available template provided by SwiftOnSecurity [13]. This configuration template is an industry standard and will be tailored to capture relevant security events while avoiding unnecessary noise, ensuring the dataset contains useful and manageable information.

Normalization will involve standardizing log entries to make the dataset consistent and easily processed by machine learning algorithms. Missing data points, such as file paths in network-related events, will be filled with placeholder values like "NULL" or zero. For Sysmon Event IDs one-hot-encoding will be used to clearly represent categories as distinct features, allowing machine learning models to learn patterns without mistakenly interpreting categories as numeric or ordinal values. Other normalization activities can also be involved based on the form of the final dataset and need for it. This will ensure each log entry contains the same structure, facilitating consistent analysis.

The final processed dataset will be divided into two sets: a training set and a test set. The training set will be used to train machine learning models, while the test set will remain unseen by models during training. This split will ensure an unbiased evaluation of model performance. Both subsets will maintain the ratio of benign to malicious events, ensuring representative samples of each category.

Through these preprocessing steps, raw Sysmon logs will be transformed into a clean and learnable format. This kind of preprocessing is very important - a powerful machine learning model can only be as effective as the data it is given.

3.5 Model Selection

The choice of machine learning models in this study is guided by the nature of the Sysmon log dataset, the binary classification objective (malicious vs. benign activity), and the limitations of the computational environment (specifically, a single desktop machine without access to high-performance computing infrastructure). To accommodate these constraints while maintaining analytical rigor, two models were selected to represent distinct methodological approaches: one non-context-aware and one context-aware.

Random Forest (RF) is chosen as the non-context-aware model. RF is an ensemble learning method based on decision trees and is well-suited for structured tabular data such as event logs. It offers a favorable balance between accuracy and efficiency, and includes

built-in mechanisms for assessing feature importance, which aids in model interpretability. Due to its low computational requirements and strong performance in prior studies on intrusion detection, RF serves as a reliable baseline for detecting threats based solely on individual log entries, without temporal or contextual awareness [5].

To capture the sequential nature and temporal dependencies inherent in log data, a more advanced, context-aware model based on Long Short-Term Memory (LSTM) networks will also be utilized. LSTMs are a type of recurrent neural network designed to handle time-series or sequential input, making them well-suited for modeling patterns across sequences of system events. By analyzing logs in their temporal context, the LSTM model can detect more subtle or persistent indicators of malicious behavior that may not be evident from isolated entries [14].

3.6 Model Evaluation

Evaluating the performance of the selected machine learning models is a key part of the methodology. Since the task is a binary classification problem (benign vs. malicious), appropriate evaluation metrics must be used to quantify how well the models can distinguish between these two classes.

Given the scope of this project and the goal of proving concept feasibility rather than developing a production-grade solution, the following basic yet effective evaluation metrics will be used:

Accuracy: Measures the proportion of correctly classified events (both benign and malicious) out of all predictions. While intuitive, accuracy can be misleading in imbalanced datasets where one class dominates.

Precision: Represents the percentage of correctly predicted malicious events out of all events predicted as malicious. High precision indicates low false positives.

Recall (Sensitivity): Indicates the proportion of actual malicious events that were correctly identified. High recall suggests fewer false negatives.

F1-Score: The harmonic mean of precision and recall. This metric provides a balance between the two and is especially useful when the dataset is moderately imbalanced, as is expected here.

More advanced metrics like ROC-AUC or log loss are intentionally omitted to keep the evaluation accessible, relevant, and easy to interpret given the limited scale and computational setup.

Each model (Random Forest and LSTM) will be trained on the training set and then tested on the hold-out test set described earlier. The performance metrics will be calculated based on this evaluation.

4 Implementation

Following chapter focuses on the practical part of the research and touches upon dataset generation and preprocessing, proof-of-concept model implementation and evaluation.

4.1 Dataset generation

To train and evaluate malicious activity detection models, a comprehensive dataset of Sysmon logs was generated on synthetic activity. This dataset includes both benign activity logs (simulated normal user behavior) and malicious activity logs (simulated attack behaviors).

As mentioned before, there was no expectation to produce a fully realistic dataset, rather one that would mimic a production environment benign activity as close as possible, while also being suitable for the objectives of this practical exercise.

The following subsections detail the process of generating the benign and malicious event data, labeling the events, and preprocessing the data for modeling.

4.1.1 Benign Activity Generation

Benign system activity was produced by simulating realistic user behavior on a Windows virtual machine using the GHOSTS user simulation framework [15]. GHOSTS framework was chosen, because it can autonomously drive common applications and actions (e.g., creating documents, browsing websites, using email) to mimic a human user's routine operations and it is also widely used in various cybersecurity exercises. A Windows 11 virtual machine was configured with the GHOSTS agent to perform predefined benign scenarios. This ensured that all generated events were collected via Sysmon (System Monitor) as typical background and user-initiated activities. Sysmon was installed with a configuration to log relevant events (process creation, file access, network connections, etc.) with a previously mentioned SwiftOnSecurity filter [13], slightly modified to better suit the needs of this work.

Several distinct activities were scripted to run in parallel, providing diversity in benign events. The activities included: file creation, modification, saving and other usual activities with Microsoft Office365 applications, specifically Excel, PowerPoint and Word. Internet browsing was also configured, simulating normal user behavior using Google Chrome.

The outcome of the GHOSTS-driven simulation was a rich set of Sysmon event logs reflecting normal usage patterns in corporate user endpoint.

4.1.2 Malicious Activity Generation

To generate malicious events, the Atomic Red Team (ART) framework was employed, specifically the Invoke Atomic Red Team module, that allows powershell scripting opportunities for test running automations. Atomic Red Team was selected as it provides a library of scripted attack techniques mapped to the MITRE ATT&CK framework, allowing reproducible execution of a wide range of adversarial behaviors that can be easily automated, as well as for being a widely used tool in industry [16]. The attacks were executed on the same Windows VM in a controlled manner.

Due to the bachelor's thesis time and scope constraints, as well as computational resource constraints, it was decided to focus on a single MITRE ATT&CK technique, that is related to generated user benign activity. The chosen technique was T1053.005 - Scheduled Task/Job: Scheduled Task, with Atomic Test #5 - Task Scheduler via Visual Basic for Applications(VBA). The test allowed choice of applications invoking the VBA either Word or Excel, which aligns with the applications that were used for benign user activity simulations.

A custom automation script (ART_automation.py¹) was developed to systematically execute the test with random intervals of items and random selection of invoking applications. The automation handled any required prerequisites or cleanup between tests, enabling unattended batch execution of attacks. The intervals between test executions were configured to be random and unrealistically short to intentionally oversample the

¹ Source Code (GitHub):

https://github.com/EphemeralShine/SysmonML/blob/main/scripts/ART_automation.py

malicious activity for reasons discussed in the previous chapter. The used interval was a randomly selected value between 15 and 30 seconds after execution and cleanup of each test run. The invoking application (Word or Excel) was also selected randomly.

Each test, when executed, generated a series of Sysmon events indicative of malicious activity. For instance, the selected test generated a variety of different Events, including Process Creation (Event ID 1), Network connection (Event ID 3), File creation (Event ID 11), Registry Modification (Event ID 13), DNS query (Event ID 22).

The execution of the Atomic Red Team tests yielded numerous malicious Sysmon log events. All such events were recorded in Atomic Red Team's log that was later used for labeling. The volume of malicious events was smaller compared to benign events (since attacks are short-lived bursts of activity), but they provide critical examples of attack footprints in the dataset.

4.1.3 Data Labeling

After collecting the raw logs from benign and malicious simulations, each event in the combined log dataset was labeled as benign or malicious (0 or 1). Labeling was performed by correlating process-related events with the known benign or malicious origin of the process by a custom Python script (`data_label.py`¹).

The initial plan involved labeling based on GUIDs of the malicious processes logged in Atomic Red Team's log, however this is where first problems arose. The GUID logging implementation of Atomic Red Team was bugged and did not log correct GUIDs and instead for each execution of the test logged exactly the same GUID. As the solution to fix this problem was not found and setting up a custom logging solution would involve significant resources, it was decided to find an alternative solution.

Labeling based on process IDs that were correctly logged by Atomic Red Team alone was unreliable, since within a period of time process IDs will be cycled due to the limited number of available values. After careful inspection of the generated dataset, it was

¹ Source Code (GitHub):

https://github.com/EphemeralShine/SysmonML/blob/main/model/general_data_manipulation/data_label.py

discovered that benign events, where powershell being either an image or a parent image were extremely rare, which is logical as activity generated by GHOSTS didn't involve usage of powershell. Following this discovery, the decision was made to label events based on process IDs correctly logged by Atomic Red Team, and as an additional safeguard for repetitive Process IDs use a check, whether image or parent image of the event is powershell, making labeling of benign events as malicious extremely unlikely. Of course, this solution is nowhere near suitable for larger enterprise level projects, however the goal of this practical task was a proof-of-concept model and such a solution seemed the most optimal in terms of time spent and result achieved.

The final distribution of events consisted of 132618 benign events and 39041 malicious ones, as also shown on the pie chart (Figure 1), showcasing oversampled amounts of malicious events as intended before.

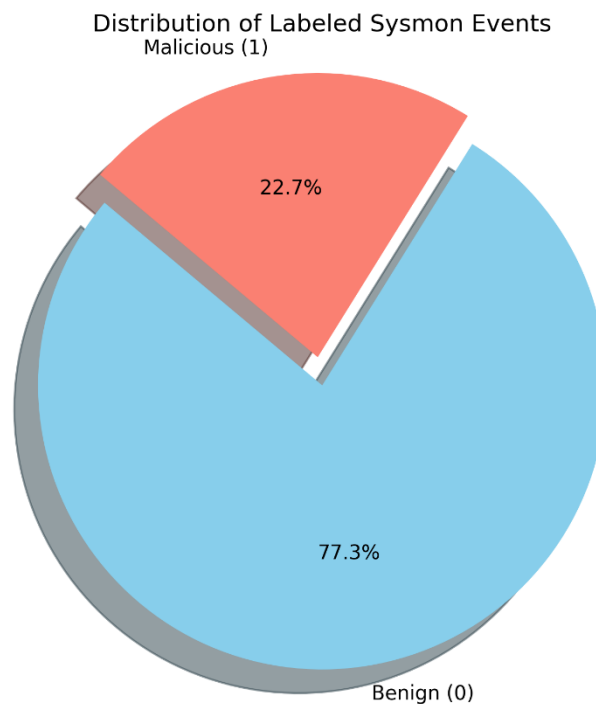


Figure 1. Distribution of Labels

4.1.4 General Dataset Preprocessing

Following the data collection and labelling steps, the Sysmon event logs required additional general preprocessing to ensure compatibility with machine learning models. Since the original event logs were semi-structured and many important attributes were nested within the “Message” field, this step focused on flattening the data, standardizing missing values, and eliminating redundant fields. Custom Python script (`data_normalise.py`¹) was also implemented for this task.

Each log entry in the dataset includes a Message column, which encapsulates structured Sysmon event attributes (e.g., Image, CommandLine, Hashes) as a string-encoded dictionary. This field was programmatically parsed using Python to safely convert the string representation into native Python dictionary objects.

After parsing, the extracted dictionaries were flattened into tabular format using the Pandas library. This step enabled direct access to individual attributes as standard DataFrame columns.

Certain key fields like ProcessId, ParentProcessId, and Image were already present in the original log entries. These fields were excluded from the parsed Message dictionary to prevent redundancy.

Columns containing numerical identifiers (e.g., ProcessId) were often cast to float during import, resulting in entries like "11328.0". To restore proper formatting, these values were converted to strings with any trailing .0 removed.

To ensure consistency and avoid issues with model training, all missing or undefined values (NaN) were replaced with the string "NULL". This ensured that categorical encoders and models could uniformly interpret unknown or missing values.

The cleaned metadata columns and the structured message fields were concatenated into a single DataFrame, resulting in a fully normalized representation of each Sysmon event.

¹ Source Code (GitHub):

https://github.com/EphemeralShine/SysmonML/blob/main/model/general_data_manipulation/data_normalise.py

The final dataset was saved as a separate file and used in subsequent feature selection and modelling stages.

This preprocessing step was essential to bridge the gap between raw Sysmon event data and machine learning-ready input, transforming nested log entries into a consistent and analysable format.

4.2 Models Training and Evaluation

This section describes the training of two models for detecting malicious activity in Sysmon logs: a Random Forest classifier and an LSTM-based neural network. The Random Forest serves as a baseline, treating each event independently, whereas the LSTM model is context-aware, operating on sequences of events.

4.2.1 Random Forest

To establish a non-context-aware detection baseline, a Random Forest (RF) classifier was trained on the preprocessed Sysmon event dataset. This model treats each event independently, relying solely on static attributes without considering the temporal sequence of events. Training was done using custom Python script (RF_model.py¹).

Before training, fields that would not provide meaningful information to non-context aware model (e.g., Process IDs, GUIDs, timestamps, IP addresses, hashes, etc.) were dropped from the dataset.

The remaining categorical features were processed through a combination of label encoding and one-hot encoding (Sysmon Event IDs) to create a clean, machine learning-ready dataset.

The resulting data was split into training and testing sets, in an 80 to 20 margins, preserving the original distribution between benign and malicious events.

¹ Source Code (GitHub):

https://github.com/EphemeralShine/SysmonML/blob/main/model/rf/RF_model.py

A Random Forest classifier was trained using all available features. The model was configured with 100 trees, balanced class weighting to address class imbalance, and a fixed random seed for reproducibility.

The performance of the model on the test set is summarized in Table 1 and visualized in confusion matrix (Figure 2):

Table 1. Random Forest Classification Report – All Features

Class	Precision	Recall	F1-Score	Support
Benign (0)	0.98	0.93	0.96	26524
Malicious (1)	0.80	0.94	0.86	7808
Accuracy			0.93	34332

Confusion Matrix (Random Forest - All Features)

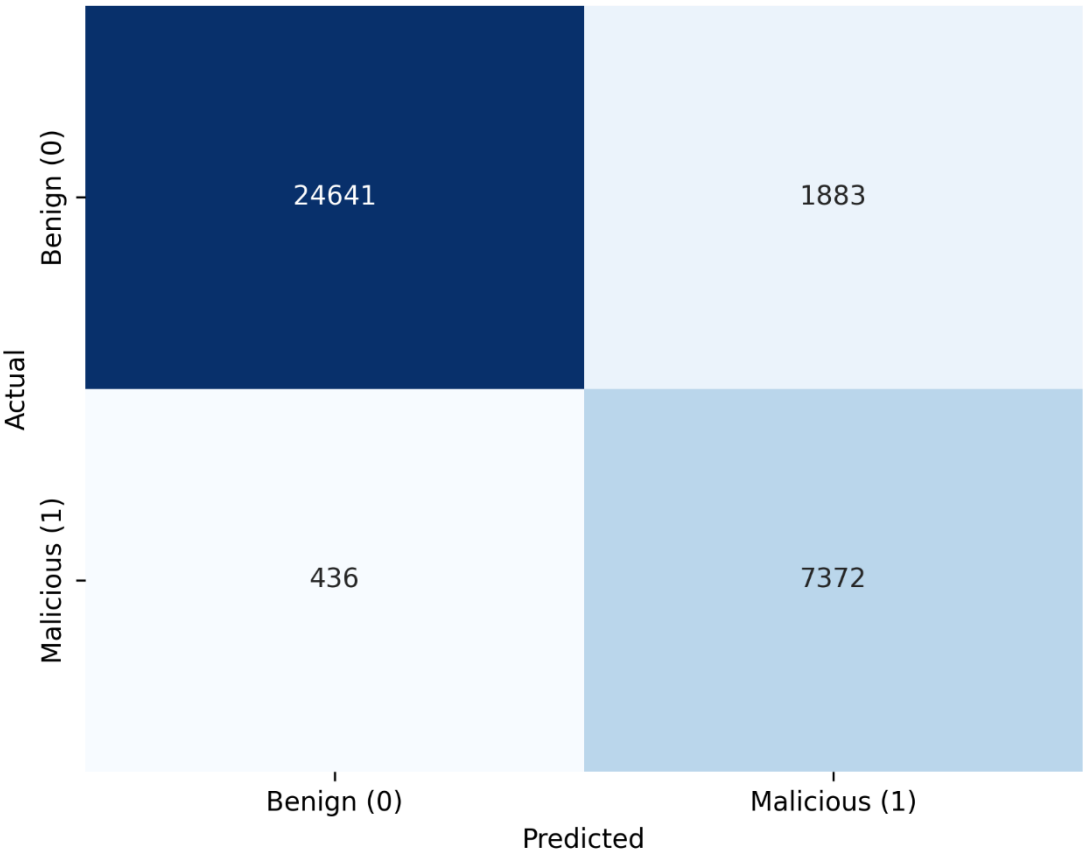


Figure 2. Random Forest Confusion Matrix - All Features

The results showed that while benign events were generally classified correctly, a notable number of benign events were misclassified as malicious, contributing to lower precision.

Feature Importance Analysis showed that a small subset of fields dominated the model’s decision-making process. The top features ranked by importance can be seen in a Figure 3:

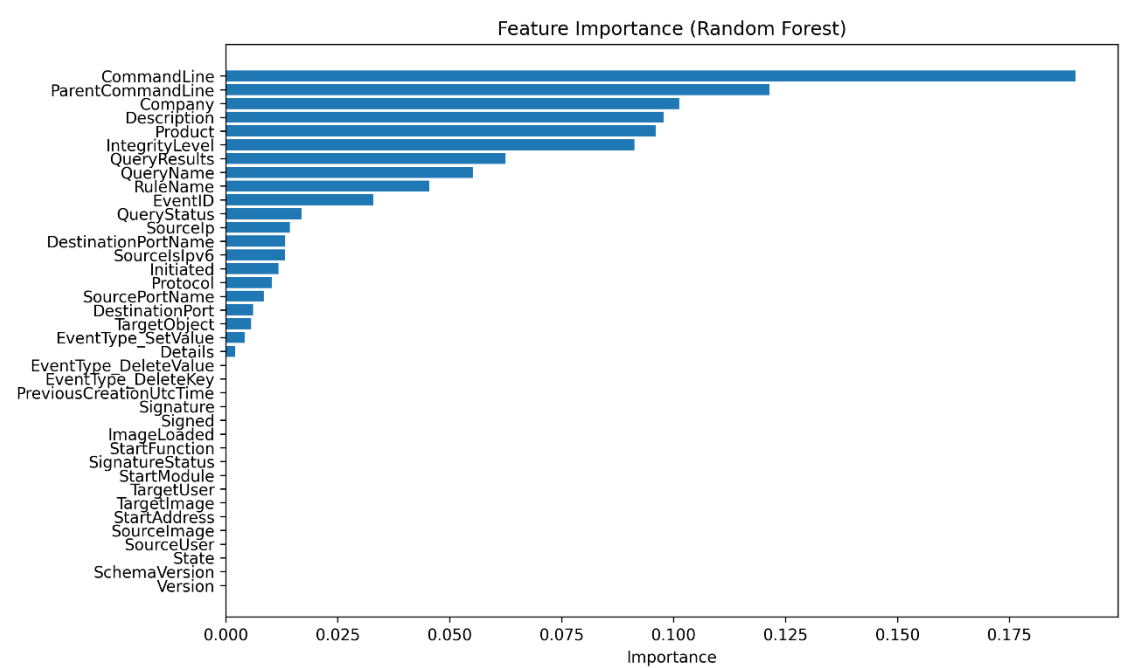


Figure 3. Feature Importance Chart for Random Forest Classifier

The heavy dependence on CommandLine and ParentCommandLine fields highlights a significant limitation - models based solely on static fields risk brittleness, as adversaries can easily manipulate these attributes to evade detection. This indicates that although Random Forest achieved good numeric results, its underlying decision basis may not be robust against real-world adversarial behavior.

To assess how much predictive performance could be retained using only the most important features, the Random Forest was retrained using the top six features based on the feature importance metric presented above.

The performance of the reduced model is summarized in Table 2 and visualized in confusion matrix (Figure 4):

Table 2. Random Forest Classification Report – Top 6 Features

Class	Precision	Recall	F1-Score	Support
Benign (0)	0.93	0.96	0.94	26524
Malicious (1)	0.84	0.74	0.79	7808
Accuracy			0.91	34332

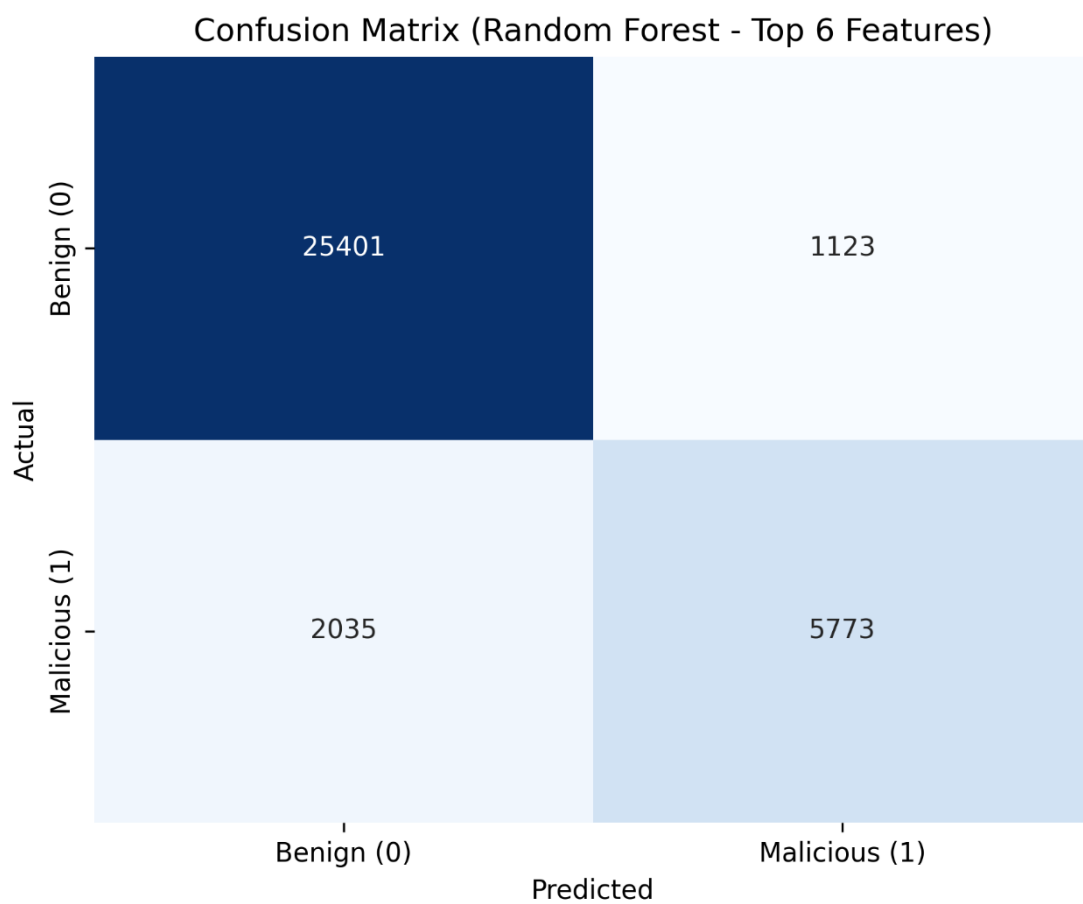


Figure 4. Random Forest Confusion Matrix - Top 6 Features

After retraining, the model maintained relatively high precision but suffered a noticeable decline in recall.

This result suggests that although the top features were highly influential, restricting the model to these alone led to a reduced ability to identify all malicious events.

The lower recall also supports the concern that overreliance on a few static fields weakens the generalization capability of the model.

4.2.2 LSTM

To address the limitations of context-agnostic models like Random Forest, a Long Short-Term Memory (LSTM) neural network was implemented. LSTM, is well-suited for modeling sequences, making it perfect for learning behavioral patterns over time in log data. Its architecture enables it to retain memory over long sequences, which is crucial in identifying subtle, time-dependent malicious behaviors.

In this study, the LSTM model was trained on sequences of Sysmon events generated using a sliding window approach. Each sequence consisted of 15 consecutive events with a stride of 1. A sequence was labeled as malicious (class 1) if it contained 3 or more malicious events ($\geq 20\%$). Such values were chosen due to the nature of the built lab environment, as it was averaging 80-100 events per minute and the execution of test attack taking approximately 5 seconds, with interval between them 15 and 30 seconds.

The LSTM specific preprocessing was done with a custom Python script (`data_preprocess_lstm.py`¹). The total number of sequences was equal to 171,644 and the distribution between malicious and benign sequences ended up being: 84795 benign and 86849 malicious, achieving a balanced dataset.

The raw data was first label-encoded to convert all categorical fields into numerical values. All feature vectors were then normalized, and sequences were split into training and testing sets using an 80/20 split to preserve the label distribution, which resulted in 137315 training samples and 34329 testing samples.

¹ Source Code (GitHub):

https://github.com/EphemeralShine/SysmonML/blob/main/model/lstm/data_preprocess_lstm.py

Training was done using custom Python script (train_lstm.py¹). Model architecture was described in custom Python class (lstm_model.py²) and consisted of a 2-layer LSTM with a hidden size of 128 and dropout set to 0.3 to prevent overfitting. Training was conducted for 15 epochs, achieving a smooth and consistent downward trend in loss values. No sudden increases or large fluctuations in loss were observed, indicating stability and absence of clear overfitting at these stages. As demonstrated in the loss plot (Figure 5), by epoch 14, the model had nearly reached its optimal performance, with only minor improvements observed thereafter.

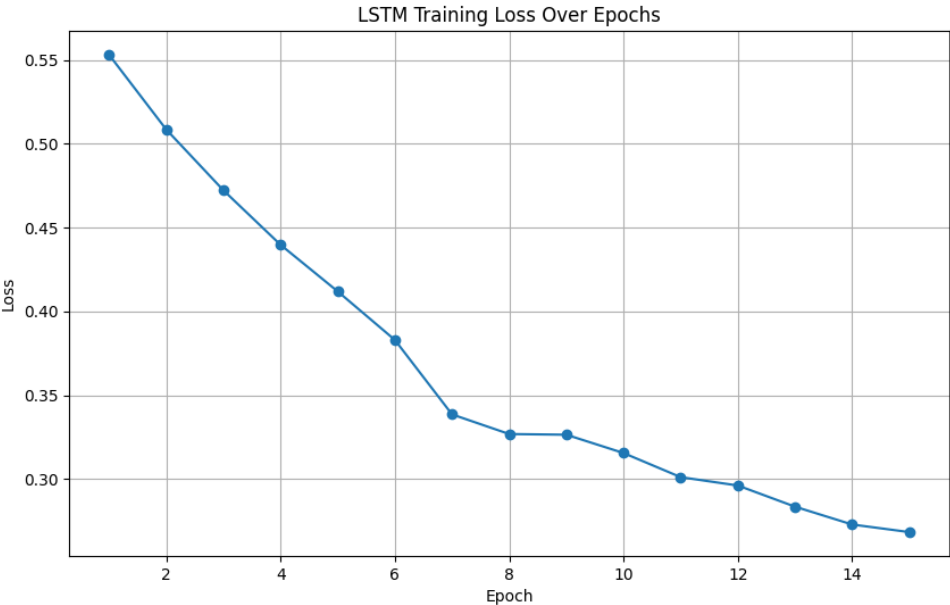


Figure 5. Training Loss Reduction Over Epochs for LSTM Model

The final evaluation on the test set produced results shown in below table (Table 3) as numeric values, the visualization of the results includes: precision-recall curve (Figure 6) and confusion matrix (Figure 7).

¹ Source Code (GitHub):
https://github.com/EphemeralShine/SysmonML/blob/main/model/rf/RF_model.py

² Source Code (GitHub):
https://github.com/EphemeralShine/SysmonML/blob/main/model/lstm/train_lstm.py

Table 3. LSTM Classification Report

Class	Precision	Recall	F1-Score	Support
Benign (0)	0.92	0.85	0.89	16959
Malicious (1)	0.86	0.93	0.90	17370
Accuracy			0.89	34329

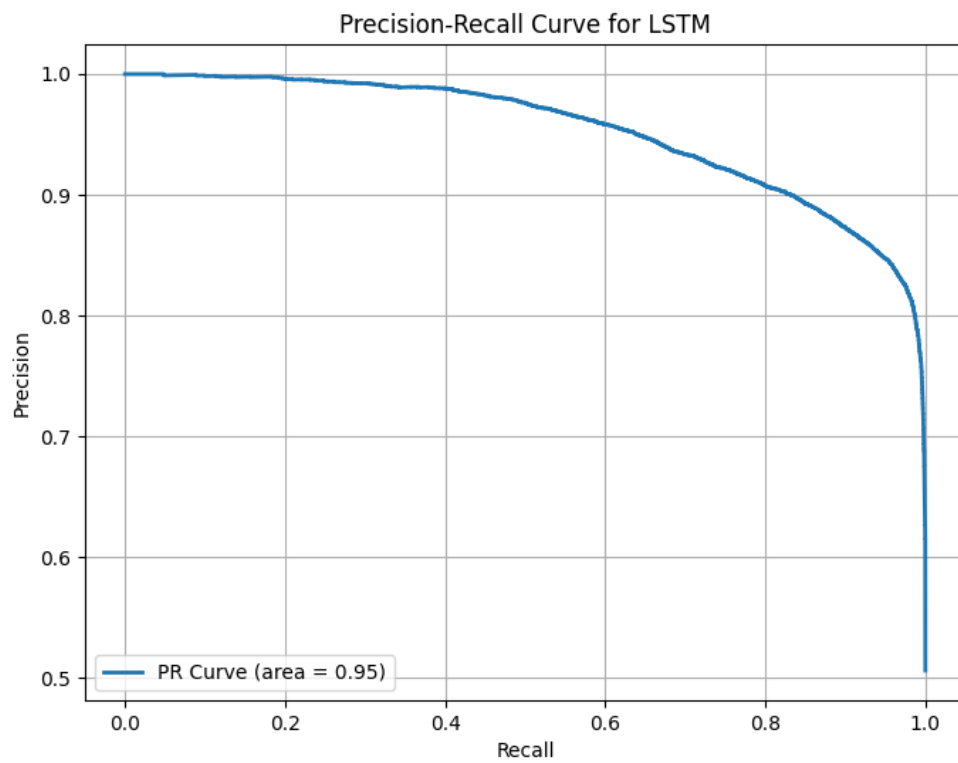


Figure 6. Precision-Recall Curve for LSTM Model

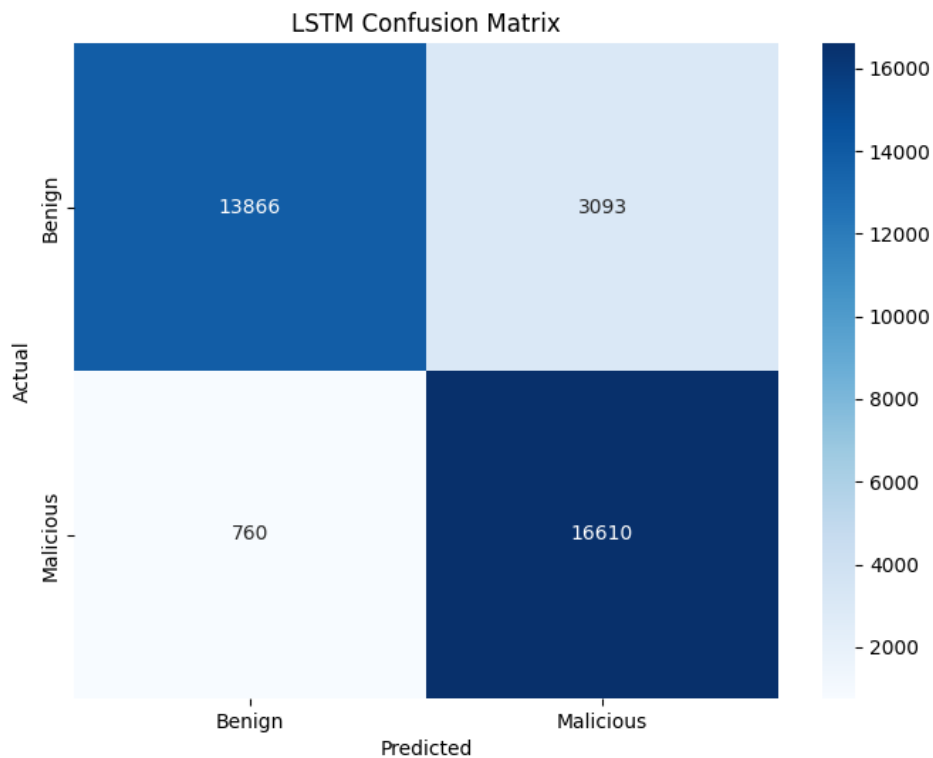


Figure 7. LSTM Confusion Matrix for Benign vs. Malicious Classification

These results indicate robust and balanced performance, with strong precision and recall metrics demonstrating the LSTM model's capability in effectively distinguishing between benign and malicious sequences. The inclusion of temporal context changed the model's predictive power by making it predict based on behavioral patterns rather than static fields.

Recall of 0.93 demonstrates the model's effectiveness in capturing most malicious activities, minimizing false negatives. The F1-score provides a balanced measure of the model's accuracy, and a score of 0.90 for malicious sequences indicates a strong overall performance. The benign class also achieved high precision and recall, reflecting the model's balanced handling of both classes without a bias toward malicious detection. Although value of 0,85 for recall of benign class signals that 15% of benign sequences were classified as malicious by the model, which in production environment would end up in false positive alerts.

The overall accuracy of 0.89 signifies that the model correctly classified 89% of all sequences in the test set.

4.2.3 Results Interpretation

The results of the experiments highlight the fundamental differences between static and sequence-based machine learning approaches for Sysmon event log analysis.

The Random Forest model, trained on individual events and static attributes, demonstrated strong performance within a controlled environment. However, its reliance on a small set of fields, particularly command-line arguments, underlines a critical weakness. Static features are relatively easy to manipulate or obfuscate by attackers, limiting the model's effectiveness in dynamic settings. Furthermore, treating events independently means Random Forest is unable to capture multi-step behaviours or contextual dependencies across events.

In contrast, the LSTM model leveraged the temporal structure of Sysmon events to reason about sequences of activities rather than isolated data points. By modelling how behaviours unfold over time, the LSTM was able to achieve more balanced detection across benign and malicious classes, offering stronger generalization. Its behavioural focus makes simple field obfuscation attacks less effective and improves resilience in real-world environments.

Overall, while Random Forest served as an effective static baseline, the LSTM approach demonstrated more impressive detection capabilities by showing promising results in classifying and learning not on individual events, but rather on sequences of events.

5 Discussion

This thesis explored machine learning-based detection engineering as an alternative and enhancement to traditional rule-based approaches for analysing Sysmon event logs. In this chapter broader topics that this work touched upon will be discussed based on the gathered data throughout the research. The topics include how machine learning-driven detection differs from rule-based detection engineering, how synthetic data influences detection development, and ultimately, how ML approaches can complement, rather than replace classical rule-based systems.

5.1 Detection Engineering: Rule-Based vs Machine Learning

Rule-based detection engineering has long been the foundation of security monitoring. Analysts craft rules based on known malicious behaviors such as specific command-line invocations or file path patterns to detect threats. The advantage of this approach lies in its precision for known attack patterns: when a rule triggers, it often does so with high confidence. However, the limitations are equally well known. Static rules are brittle. They require continuous maintenance, suffer from blind spots when facing novel attacks, and can flood analysts with false positives if too broadly defined.

In contrast, the ML-based approach demonstrated in this thesis allows for adaptive and pattern-based threat detection. Instead of predefining signatures, machine learning models, especially when trained on a diverse dataset, learn to generalize behaviors that correlate with malicious activity. This enables models to detect unseen variations of attacks that rule-based detections would likely miss. Furthermore, context-aware models like LSTM introduce an important improvement over rules - temporal understanding. They can capture sequences of events indicative of compromise, rather than single isolated indicators. While these capabilities highlight the potential of machine learning for detection engineering, the outcomes of this thesis must also be interpreted within the context of their experimental nature.

The proof-of-concept models trained as a part of this thesis showed that machine learning can indeed be applied effectively to detection engineering tasks. However, their effectiveness is fundamentally tied to the quality, diversity, and realism of the data they are trained on. While the models demonstrated promising performance on the synthetically generated dataset, they remain prototypes that are unlikely to fully translate to a production environment without adaptation. The constraints of synthetic activity, limited attack diversity, and controlled lab conditions mean that real-world deployments would expose models to a far broader range of behaviours and noise. Thus, the models developed in this thesis should be viewed as experimental proofs of concept: validating that machine learning can assist detection engineering, while also highlighting the necessity for further work in dataset realism, continuous retraining, and integration with traditional rule-based systems for operational use.

While synthetic datasets allowed for controlled experimentation, their inherent simplicity compared to live enterprise environments means that real-world validation remains essential. Combining real-world log data with synthetically generated malicious activity could help bridge this gap, offering models exposure to more realistic noise, user behaviour, and edge cases. Without such enrichment and careful adaptation, machine learning models risk inheriting biases from their training data and may struggle with the complexity and unpredictability of production environments.

It is important to recognize that machine learning is not a silver bullet. ML models can suffer from biases in the training data, may lack explainability, and in real-world deployments still require careful tuning, monitoring, and retraining. Unlike rule-based systems where a detection can be easily audited and explained, ML models often operate as black boxes, making post-alert investigations more challenging. And even after all these disadvantages, obtaining a proper training dataset is still a huge challenge, a solution of which directly translates to the quality of the model.

Considering this, rather than replacing rule-based detections, ML models should be seen as enhancements. Rule-based detections offer clarity and immediate actionability, while machine learning can expand coverage into gray areas where rules are too rigid or too labor-intensive to maintain. In practice, combining both approaches leads to a stronger detection engineering strategy: using rules for high-fidelity known behaviors, and ML models to identify emerging or less structured threats.

5.2 The Role of Synthetic Data Generation

Synthetic data played a big role in this research. Because real-world labelled openly available Sysmon datasets were not found by the author due to privacy, compliance, and operational security constraints, synthetic generation provided a necessary workaround. Tools like GHOSTS and Atomic Red Team allowed the construction of a controlled, labeled dataset containing both benign and malicious behaviors.

Synthetic datasets enable rapid iteration and testing of ML models. They allow for focusing on specific adversary techniques, balancing class distributions, and injecting a wider variety of examples than might naturally occur. In a real-world enterprise dataset, malicious events are very rare compared to benign noise; training models directly on real-world distributions without synthetic oversampling would make the learning process extremely difficult.

In addition to serving as a standalone training source, synthetic data can also be used to extend and enrich real-world datasets as mentioned before. In practical deployments, organizations can supplement their real Sysmon logs with synthetically generated events that cover rare, emerging, or adversarial behaviors. By doing so, machine learning models are exposed to a broader spectrum of attack patterns, improving their ability to generalize. Synthetic augmentation ensures that models are not just trained on historically observed threats but are also prepared for future scenarios. This proactive approach can increase the robustness of security detection systems and process of dataset acquisition.

However, while synthetic augmentation is powerful, it must be carefully validated to avoid introducing unrealistic artifacts that could bias model training. The fidelity of the synthetic data - its ability to mimic the structure and variability of real-world activity - remains a critical success factor.

In short, synthetic data is not merely a shortcut for unavailable datasets, but a strategic tool for strengthening and future-proofing machine learning-based detections.

5.3 Conclusion of Discussion

The results of this work suggest that ML-based detections are not meant to replace rule-based detections for now, but to extend and strengthen them.

Rule-based detections remain essential for catching well-known, deterministic threats with high confidence and explainability. They are simple to audit, quick to respond to changes, and critical for compliance requirements.

Meanwhile, ML-based detections excel at catching what rules miss - complex behaviors, subtle anomalies, and novel attack patterns that would require an impractical number of manual rules to detect.

Moreover, the combined use of synthetic data generation and machine learning might be a prominent approach for prototyping, extending, and maturing detections before they are validated against production data.

To conclude the author's discussion, this thesis demonstrates that the future of detection engineering is hybrid: a collaborative system where rule-based and machine-learning detections work together, amplifying each other's strengths and covering each other's weaknesses.

6 Limitations, challenges and future work

While the implementation of machine learning models for Sysmon event log analysis demonstrated promising results, several limitations were encountered that shape the interpretation of outcomes. At the same time, these challenges provide valuable lessons and opportunities for future research.

6.1 Limitations and Challenges

1. The dataset was generated entirely through synthetic behaviour for both benign and malicious activity. While this approach enabled controlled experimentation and ensured access to labelled data, it lacks the full variability, noise, and complexity of live production environments. As a result, the models trained in this thesis represent strong prototypes but may require significant adaptation and retraining before practical deployment.
2. Although careful strategies were applied for labelling malicious events, the reliance on heuristic methods (e.g., correlating process IDs and PowerShell usage) introduces a margin of error. Inaccurate labels, even if rare, can impact model performance and highlight the difficulties of building high-quality cybersecurity datasets at scale.
3. All experimentation was conducted using a single workstation with limited computing power. This restricted dataset size, model complexity, hyperparameter tuning, and training iterations. In future work, access to larger computational resources would enable deeper experimentation, particularly with more complex architectures.

Despite these limitations, the research successfully established a solid foundation. A full framework for synthetic data generation, preprocessing, feature engineering, model training, and evaluation was developed and validated. This structure now provides a repeatable framework that can be expanded and refined in future projects.

6.2 Future Work

Building upon the groundwork laid by this research, several areas for future exploration are suggested:

1. Future research should focus on developing techniques to intelligently augment real-world Sysmon logs with synthetic malicious activities. This hybrid approach would allow for building richer and more balanced training datasets while maintaining the authenticity of benign behaviours. Careful synthetic injections can help expose models to rare or emerging attack patterns that are difficult to capture in production environments without introducing artificial artifacts that bias model learning.
2. Expanding the scope to cover multiple MITRE ATT&CK tactics and techniques would enrich the training data and improve the model's capability to detect diverse threats.
3. Applying explainable AI (XAI) techniques to both Random Forest and LSTM models would help security analysts understand machine learning-based detections.
4. Investigating the operational challenges of deploying ML-based detections into live SIEM or EDR systems, including handling false positives in real-time.
5. Developing a more universal labelling solution that would suit wide variety of attacks.

In summary, while this work was limited by scope and resources, it successfully built a research platform capable of supporting more ambitious future detection engineering projects.

7 Conclusion

This thesis explored the use of supervised machine learning models for enhancing detection engineering through the analysis of Sysmon event logs. Through a combination of synthetic dataset generation, feature extraction, model training, and evaluation, a full proof-of-concept framework was developed and tested.

Both the Random Forest and LSTM-based models showed that machine learning can be applied effectively to correlate Sysmon logs and distinguish malicious from benign activity. Random Forest provided strong baseline performance with individual event classification, while LSTM demonstrated the power of temporal context modelling in improving threat detection quality.

This work set out to explore whether supervised machine learning models can effectively enhance detection capabilities. Specifically, it sought to answer the research question: *“What is the architecture that enables supervised machine learning models to be effectively developed and trained to detect malicious activity patterns that might evade traditional rule-based detection approaches, given the significant constraints of limited and often imbalanced real-world labelled datasets?”*. Through the design and evaluation of a controlled, synthetic data generation framework, the implementation of both Random Forest and LSTM models and subsequent discussion, this question was addressed in a practical, measurable way.

Importantly, this research emphasized that machine learning is not a replacement for traditional rule-based detection engineering, but rather a complementary enhancement. ML models can expand detection capabilities into areas too complex, dynamic, or subtle for static rules alone. However, they also introduce new challenges, including explainability issues, sensitivity to data quality and its complex acquisition.

By designing a synthetic dataset framework, and training two distinct model architectures on it, this thesis laid down a solid foundation for future advances. The models developed here serve as early prototypes, demonstrating feasibility but also clearly illustrating the critical role of data realism and operational integration in practical deployments.

Future efforts should aim to build on this platform by combining real-world and synthetic data, expanding coverage of adversary behaviour, and improving the transparency and maintainability of machine learning models in security operations.

Ultimately, this work affirms that the future of detection engineering lies in hybrid approaches: leveraging the precision and explainability of static rules, while embracing the adaptability and scalability of machine learning - a path that promises more resilient, intelligent, and efficient threat detection systems.

References

- [1] Microsoft, "Sysmon - Windows Sysinternals," Microsoft Learn, 2024. [Online]. Available: <https://learn.microsoft.com/en-us/sysinternals/downloads/sysmon>. [Accessed: 07-Mar-2025].
- [2] J. Ispahany, M. R. Islam, M. A. Khan, and M. Z. Islam, "A Sysmon Incremental Learning System for Ransomware Analysis and Detection," arXiv preprint arXiv:2501.01089, Jan. 2025.
- [3] M. J. Kruczkowski and E. Niewiadomska-Szynkiewicz, "Comparative Study of Supervised Learning Methods for Malware Analysis," J. Telecommun. Inf. Technol., vol. 2014, no. 4, pp. 24–33.
- [4] O. O. Olateju, S. U. Okon, U. T. I. Igwenagu, A. A. Salami, T. O. Oladoyinbo, and O. O. Olaniyi, "Combating the Challenges of False Positives in AI-Driven Anomaly Detection Systems and Enhancing Data Security in the Cloud," Asian Journal of Research in Computer Science, vol. 17, no. 6, pp. 264–292, Jun. 2024.
- [5] J. Doe and M. Smith, "Advancements in Random Forest Algorithms for Intrusion Detection Systems," Journal of Cybersecurity Research, vol. 15, no. 2, pp. 100-110, 2023.
- [6] A. Kumar, B. Singh, and C. Sharma, "A Comparative Analysis of Support Vector Machine and K-Nearest Neighbors Models for Network Attack Traffic Detection," International Journal of Computer Applications, vol. 182, no. 45, pp. 25-30, 2023.
- [7] M. Du et al., "DeepLog: Anomaly Detection from System Logs through Deep Learning," ACM SIGSAC Conference on Computer and Communications Security, pp. 1285-1298, 2017.
- [8] D. Levshun and I. Kotenko, "Artificial Intelligence for Security Event Correlation: A Review," IEEE Access, vol. 11, pp. 7587-7606, 2023.
- [9] N. Smiliotopoulos, E. Athanasopoulos, and S. Ioannidis, "Detection of Lateral Movement in Windows Sysmon Operational Logs Using Machine Learning," Computers & Security, vol. 127, 2023.
- [10] Elastic Security, "Machine Learning in Elastic Security," Elastic documentation, 2023 Available: <https://www.elastic.co/elasticsearch/machine-learning> [Accessed: 24-Mar-2025].
- [11] ManageEngine, "User and Entity Behavior Analytics (UEBA)," ManageEngine EventLog Analyzer, 2024 Available: <https://www.manageengine.com/log-management/ueba/user-and-entity-behavior-analytics-software.html> [Accessed: 24-Mar-2025].
- [12] D. A. Ammara, J. Ding, and K. Tutschku, "Synthetic Data Generation in Cybersecurity: A Comparative Analysis," arXiv preprint arXiv:2410.16326, 2024.
- [13] SwiftOnSecurity, "Sysmon Configuration File," GitHub, 2024. [Online]. Available: <https://github.com/SwiftOnSecurity/sysmon-config>. [Accessed 12-Apr-2025]
- [14] F. Laghrissi, S. Douzi, K. Douzi, and B. Hssina, "Intrusion detection systems using long short-term memory (LSTM)," Journal of Big Data, vol. 8, no. 65, 2021.

- [15] CMU SEI, “GHOSTS” GitHub, 2025. [Online]. Available: <https://github.com/cmu-sei/GHOSTS>. [Accessed: 15-Apr-2025].
- [16] Red Canary, “Atomic Red Team: Adversary Emulation Library”, GitHub, 2025. [Online]. Available: <https://github.com/redcanaryco/atomic-red-team>. [Accessed: 15-Apr-2025].

Appendix 1 – Non-exclusive licence for reproduction and publication of a graduation thesis¹

I Daniil Fetissov

1. Grant Tallinn University of Technology free licence (non-exclusive licence) for my thesis “Sysmon Event Log Analysis Using Machine Learning”, supervised by Margus Ernits.
 - 1.1. to be reproduced for the purposes of preservation and electronic publication of the graduation thesis, incl. to be entered in the digital collection of the library of Tallinn University of Technology until expiry of the term of copyright;
 - 1.2. to be published via the web of Tallinn University of Technology, incl. to be entered in the digital collection of the library of Tallinn University of Technology until expiry of the term of copyright.
2. I am aware that the author also retains the rights specified in clause 1 of the non-exclusive licence.
3. I confirm that granting the non-exclusive licence does not infringe other persons' intellectual property rights, the rights arising from the Personal Data Protection Act or rights arising from other legislation.

12.05.2025

¹ The non-exclusive licence is not valid during the validity of access restriction indicated in the student's application for restriction on access to the graduation thesis that has been signed by the school's dean, except in case of the university's right to reproduce the thesis for preservation purposes only. If a graduation thesis is based on the joint creative activity of two or more persons and the co-author(s) has/have not granted, by the set deadline, the student defending his/her graduation thesis consent to reproduce and publish the graduation thesis in compliance with clauses 1.1 and 1.2 of the non-exclusive licence, the non-exclusive license shall not be valid for the period.